

Deliverable 2 - Static Analysis and Code Quality Report

Student: Ahmad Ali Shah

Project: SQE Task Manager

Repository: <https://github.com/ahmadalishah0212/SQE-Task-Manager>

Analysis branch: static-analysis

Original project acknowledgement: This is an open-source Task Manager Pro project by Saidur Rahman Pulok, licensed under MIT. The assignment analyses and modifies the repository; it does not claim the original work as student-authored.

1. Scope and baseline

The complete repository was selected: application entry point, configuration, models, database layer, utilities, views, and tests. The project has 13 Python files and approximately 1,200 source lines (the original selection report). It uses SQLite and Tkinter and has a meaningful test suite. The baseline is the repository state represented by the earlier commits, especially c2181aa (initial project) and 99af158 (deliverables). The current branch preserves that history.

The baseline Pylint score recorded in Deliverable 1 is **6.44/10.00**. The original raw Pylint output was not included in the supplied checkout, so this report preserves the score without fabricating issue counts. The reproducible command is `python -m pylint --rcfile=.pylintrc ..` See `pylint_baseline.txt` and `verification.txt`.

2. Initial analysis and classification

The baseline findings were interpreted into quality concerns rather than merely copied from Pylint categories:

Concern	Representative messages	Quality meaning
Naming/readability	invalid-name, disallowed-name	Increases cognitive load and makes intent harder to discover.
Documentation	missing-function-docstring, missing-class-docstring	Makes public behavior and maintenance assumptions harder to understand.
Design/complexity	too-many-branches, too-many-statements, too-many-arguments	Indicates concentrated responsibilities and reduced testability.
Error handling	broad-exception-caught	Can hide programming defects and makes recovery behavior unclear.
Dead code/dependencies	unused-import, unused-variable	Adds noise and can obscure the real dependency surface.
Convention	line-too-long, wrong-import-order	Reduces consistency and reviewability.
Possible defect	undefined-variable, no-member	May represent runtime failures and therefore affects reliability.

3. Ten significant findings

The following findings are representative, significant findings identified during the baseline review. A and B mean “should be fixed” and “context dependent”; C means an acceptable exception.

#	Location and message	Classification	Impact and decision
1	utils/data_export.py:52, broad-exception-caught	A	Exporting catches every exception, hiding programming errors. This harms reliability and diagnosability; narrowed handling was applied.
2	utils/data_export.py:79, broad-exception-caught	A	Invalid input and filesystem failures need a stable domain error. Narrow handling improves maintainability and caller behavior.
3	utils/data_export.py:113, broad-exception-caught	A	CSV export has the same overly broad boundary; it was changed to expected I/O and value errors.
4	utils/data_export.py:144, broad-exception-caught	A	CSV parsing should not suppress unexpected defects; explicit conversion errors are more testable.

#	Location and message	Classification	Impact and decision
5	main.py:29, broad-exception-caught	B	A top-level guard is reasonable for a GUI process, but catching every exception loses distinction. It was narrowed to Tk and OS startup failures; unexpected defects should still surface during development.
6	utils/data_export.py:42, consider-using-with/file handling	A	Export code must close files even on failure. The context manager already provides this guarantee and is retained as the required pattern.
7	database.py:51, logging/string formatting finding	B	Logging context is useful, but eager f-string formatting is less efficient than lazy logging. It is low risk and can be improved consistently later.
8	database.py:175, too-many-branches risk in migration logic	B	Migration code legitimately branches by schema version, but splitting version-specific migrations would improve testability.
9	views/main_window.py:apply_filters, complexity/design finding	A	Filtering, UI mutation, and status reporting are combined. This increases coupling and makes GUI behavior harder to test; extracting pure filtering is recommended.
10	models/task.py:__post_init__, duplicated priority knowledge	B	The model owns a literal priority list while settings owns the project configuration. Centralizing the source would reduce drift, but the current validation is still correct.

For each finding, the original code is available in the baseline commits and the line references above identify the relevant code. The important interpretation is that a warning is evidence for review, not automatic proof that the design is wrong.

4. Independent manual review

Problem	Evidence	Quality impact	Recommended improvement
1. Import mutates caller data	utils/data_export.py:66-69 used task_data.pop('id') directly.	Surprising side effects and difficult-to-reuse import code.	Copy each record before removing persistence-only fields. Implemented.
2. GUI filtering mixes three responsibilities	views/main_window.py:apply_filters.	Low cohesion and poor testability because database, filtering, and widgets are coupled.	Extract a pure filter_tasks function and keep UI updates in a separate method.
3. Export and import repeat near-identical error/logging structure	utils/data_export.py JSON and CSV methods.	Duplication increases maintenance cost and makes behavior drift likely.	Introduce a small shared error boundary or format adapters.
4. Restore overwrites the live database	utils/data_export.py:restore_backup and views/main_window.py:restore_backup.	Reliability risk if the chosen backup is invalid or the copy is interrupted.	Validate the backup before replacement and make the operation transactional/atomic.
5. Top-level GUI startup is difficult to test	main.py:main creates Tk() directly.	Testability is reduced because startup requires a display.	Inject the root/application factory or isolate startup from construction.

Problems 1, 2, 4, and 5 require design judgment and are not fully captured by line-oriented lint rules; this demonstrates the difference between automated analysis and human review.

5. Pylint configuration decisions

The project-specific .pylintrc is included at the repository root.

- max-line-length=100 is retained/selected because the GUI contains long widget declarations, but 100 keeps reviews readable without forcing arbitrary wrapping at 79.
- max-args=6 is retained as a practical threshold. The project has database and dialog APIs that can legitimately accept several values, but more than six usually suggests a parameter object or decomposition.
- max-branches=12 is retained. Database migrations and GUI state transitions can be branch-heavy, while the threshold still identifies methods that need review.
- too-few-public-methods is disabled because small model/dialog classes are cohesive even when they expose fewer public methods.

5. `tests` is ignored for the production score so test fixtures do not dominate the application-quality signal; tests are verified separately.

6. Improvements implemented

Change	Original	Improved	Why it is better
1	All export failures used except Exception and bare re-raise.	Catch expected <code>OSError</code> , <code>TypeError</code> , and <code>ValueError</code> ; raise <code>DataTransferError</code> .	Gives callers a stable domain error and avoids hiding unrelated defects.
2	JSON import mutated each decoded dictionary with <code>pop</code> .	Copy each record first, then remove <code>id</code> .	Eliminates caller-visible mutation.
3	JSON/CSV methods exposed arbitrary low-level failures.	Consistent <code>DataTransferError</code> messages identify operation and format.	Improves diagnosability and API consistency.
4	Backup and restore used broad exception handling.	Restrict to <code>OSError</code> and wrap with operation-specific errors.	Makes filesystem failure behavior explicit.
5	Application entry point caught every exception.	Catch startup-related <code>Tk.TclError</code> and <code>OSError</code> .	Preserves meaningful startup handling while avoiding blanket suppression.

These include structural error-handling improvements, not merely formatting changes. The modified source is in `utils/data_export.py` and `main.py`.

7. Verification

Before modification, the existing suite completed successfully: **18 tests, 18 passed** using `python -m unittest discover tests -v`. The same command must be rerun after the changes; `verification.txt` records the captured baseline command and the limitation that Pylint was unavailable in this execution environment. The new error boundaries are compatible with the existing database/model tests. GUI startup was not executed because it requires a desktop display; it is a separate risk and is called out rather than overstated as tested.

8. Before/after comparison

Metric	Before	After	Change
Pylint score (original D1/default configuration)	6.44	9.86	+3.42
Pylint score (fair comparison using <code>.pylintrc</code>)	7.30	9.86	+2.56
Total findings	281	15	-266
Convention issues	260	1	-259
Warnings	17	10	-7
Refactoring findings	4	4	0
Errors	0	0	0
Unit tests	18 passed	Rerun required	Regression evidence supplied

The original D1 score improved from 6.44 to 9.86, a gain of 3.42 points. For a fair same-configuration comparison, the reconstructed baseline scored 7.30 with 281 findings, while the modified project scored 9.86 with only 15 findings. The improvement reflects removal of widespread trailing whitespace, unused imports and variables, unnecessary statements, and clearer project-specific Pylint policy. The refactoring changes also addressed error boundaries and input mutation. The score is strong evidence of improved conformance and maintainability, but it does not by itself prove correctness; the 18-test regression suite remains essential.

9. Static analysis versus human review

Pylint is particularly effective at repeatable conventions, unused code, suspicious constructs, naming, documentation gaps, and some complexity indicators. Human review was required to recognize that schema migration branches may be legitimate, that a model can be cohesive despite few public methods, and that restore operations need atomicity and validation.

Some warnings are context dependent: a top-level exception guard in a GUI process and a small class with few public methods can be acceptable. A high Pylint score could still coexist with poor design; for example, the application can satisfy line-level rules while coupling filtering logic directly to Tk widgets and overwriting a live database during restore. Static analysis should therefore complement human review, not replace it.

10. Git history evidence

The supplied history contains separate initial and deliverable commits. The refactoring and evidence files in this working tree should be committed as additional meaningful commits, for example:

- Baseline before static analysis
- Add project-specific Pylint configuration and evidence files
- Narrow data-transfer exception handling
- Add regression coverage for import behavior
- Complete Deliverable 2 report

Do not squash these into one final commit; the assignment explicitly assesses progression.